Week 3 Intro to PyTorch and Beginnings of Model Building

Agenda for today:

Quick review: what have learnt last week?

- Structure of a so-called neural network (NN)
- Layers
- What is a neuron
- What are connections between neurons, and what are weights
    - Connection between one neuron in a hidden layer:
    - Sigma(linear regression incl bias)
    - This linear equation includes the parameters w (weights) and the bias
    - → parameters, and it is the objective of the learning process to find them
- What is learning: iterative adjustment of the parameters, based on "feedback" = cost function

Watch first few minutes of Grant Sanderson's 3Blue1Brown video (up until c.a. 1:47) on Gradient Descent for a recap: https://www.3blue1brown.com/?v=gradient-descent

Quick review: virtual environments, Python modules, Jupyter Notebook

What are **virtual environments**, and why use them?
-> in short, to avoid version conflicts and make sure everyone is on the same page. For more details on their rationale and how to use them, please refer to the Python document from last term.

How to use them depends on your operating system:

- Create one: **`py3`** `-m venv name-of-your-venv` (Win: for Mac, instead of py3, use `python3`)

- Activate: `name-of-your-venv/Scripts/activate` (Win; for Mac, use `source name-of-your-venv/bin/activate`)

Please make sure to set one venv up for this module. Try to stick to this one consistently. And, remember to enable scripts on your computer!
Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope CurrentUser
(type the above in PowerShell; if on a Mac, then in Terminal)

Also remember to work locally—not on OneDrive or anything like that.

What are **Python libraries/modules/packages**, and how to use them?

A package is a set of object classes, methods, and functions, possibly containing submodules, which are distributed together and specialized for particular tasks. Eg. Numpy.

We can use the **pip in-built Python package** to manage which packages are installed in our current install (whether that is the install inside a venv, or a Global Environment).

Remember, a package must be activated in a file (loaded using the `import` command) before it can be used.--> either "inside" a .py script, or "inside" a .ipynb file.

What is **Jupyter Notebook**, and why use it?

It is an interface in which you can code in a way that segments code into easier to manage chunks, which we could call code blocks. More specifically, it segments all of your code and your annotations into what are called **cells,** which can be **code cells** or **markup cells.** These cells make it super easy to break down coding tasks into chunks, work on only smaller bits at a time, and to take notes.

Why use it?
- It's very easy
- Because virtually all AI libraries and their examples exist in Jupyter Notebooks.

What is PyTorch?

PyTorch is one of the most commonly used Python libraries for training AI models. Another one you may have heard of is TensorFlow; Keras.

Installing PyTorch

Because PyTorch is a Python library, you install it the way you install packages. Here is extensive guidance on which version to install, depending on your computer:
https://pytorch.org/get-started/locally/
Note: What is CUDA?
CUDA is a parallel computing platform that allows model training to use graphics processing units (GPUs) in order to work much faster than just using your CPU. It seems you cannot download PyTorch with CUDA on a Mac, so I haven't tested it. But if you want, go ahead and try the download with CUDA. It's also not tragic if you don't use it.

On a Mac:

```
pip3 install torch torchvision
```

On Win:
Either the above (CPU), or the below (CUDA):

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

let's do the verification (=checking it works correctly) together:

```
import torch
x = torch.rand(5, 3)
print(x)
```

Getting started with learning PyTorch – using the PyTorch YT tutorial series

We're going to mostly use the PyTorch YT tutorial series, where I encourage you to watch each video as homework after class to engage with the material. However, I won't be discussing every single tutorial in the series, as some are a bit too technical/we don't have enough time. I'll be editing the ones I discuss, as well.

I also really like the PyTorch 60 Min Blitz, which is super understandable. I'll be mashing that in as well.

Today, what we want to achieve is:

- Understanding what PyTorch is, and how it works
- Getting to grips with model structure in PyTorch, which isn't going to be a full model building yet

Understanding what PyTorch is

PyTorch is a Python library—similar to NumPy, Matplotlib, Pandas, and all the other ones we have encountered. PyTorch is the library that was used to build: ChatGPT, Tesla's Autopilot (... :/), and also Hugging Face's Transformers.

Because PyTorch is a library, this means that a lot of model components are built-in classes and objects, with their pre-defined methods and operations, as well as how to use them. What are such components:

- Different types of layers (of a neural network)
- Different "activation functions" (we'll talk more about them later)
- Different optimization algorithms
- A class of object called a tensor (`torch.Tensor`)

But what is a tensor?
= essentially a glorified matrix
= basically the equivalent of NumPy's ndarray object, just with a few additional properties that make it suitable for neural networks
= a class of object (or "data type") in Python that was especially developed for use in NNs
=> its name appears also in the library of TensorFlow

Below, I've excerpted what I think is a really useful description of tensors in the PyTorch documentation:

PyTorch: Tensors

Numpy is a great framework, but it cannot utilize GPUs to accelerate its numerical computations. For modern deep neural networks, GPUs often provide speedups of 50x or greater, so unfortunately numpy won't be enough for modern deep learning.

Here we introduce the most fundamental PyTorch concept: the **Tensor**. A PyTorch Tensor is conceptually identical to a numpy array: a Tensor is an n-dimensional array, and PyTorch provides many functions for operating on these Tensors. Behind the scenes, Tensors can keep track of a computational graph and gradients, but they're also useful as a generic tool for scientific computing.

Also unlike numpy, PyTorch Tensors can utilize GPUs to accelerate their numeric computations. To run a PyTorch Tensor on GPU, you simply need to specify the correct device.

Source: https://docs.pytorch.org/tutorials/beginner/pytorch_with_examples.html

And now, let's take a look at how models work using the Intro Tutorial in the PyTorch YT series. (The .ipynb file is on Blackboard.)

Skip the bit on tensors and go directly to PyTorch Models.

Looking at the structure of a CNN, here are the things I would like you to notice and take away as we move through this file:

- The layers and layer types have their own objects/classes in PyTorch
    - Don't worry if you don't fully understand these right now!
- So too do different functions and operations, e.g. forward and backward
- There is an important component about loading and handling the data, which is not trivial
    - Here is a handy reference text for the syntax for how to handle such data: https://www.geeksforgeeks.org/python/how-to-load-cifar10-dataset-in-pytorch/
- Training the model amounts to defining the loss function, selecting a learning algorithm (=optimizer), using the .backward() function to compute gradients and then the optimizer to adjust the weights